13 · Reactive Forms

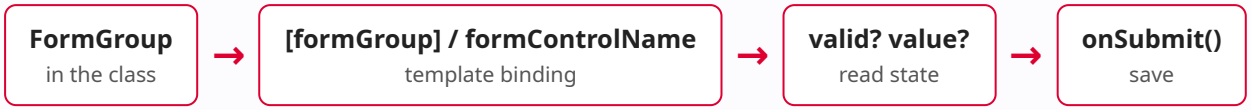
Intermediate Angular learning notes

a. Definition

In **reactive forms** you create the form model in the **component class** using `FormGroup` and `FormControl` (or the shorter `FormBuilder`). The template simply connects to it with `[formGroup]` and `formControlName`. Everything is explicit, typed and easy to test.

Use `fb.nonNullable.group({...})` to get strongly typed values. Validators are plain functions such as `Validators.required` and `Validators.email`. You can read the whole value with `form.value` or `form.getRawValue()` and react to changes with `valueChanges`.

b. Diagram



The form model lives in TypeScript; the template only binds to it

c. Syntax

```
form = inject(FormBuilder).nonNullable.group({
  name:  ['', [Validators.required, Validators.minLength(3)]],
  email: ['', [Validators.required, Validators.email]],
});

<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="name" />
</form>
```

d. Example Program

Programs/13_Reactive_Forms/reactive-form.component.ts

```
import { Component, inject, signal } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';

@Component({
  selector: 'app-root',
  imports: [ReactiveFormsModule],
  template: `
    <h3>Student Profile</h3>
    <form [formGroup]="form" (ngSubmit)="submit()">
      <input id="name" formControlName="name" placeholder="Name" />
      @if (form.controls.name.touched && form.controls.name.invalid) { <small>Name needs 3+
letters.</small> }

      <input id="email" formControlName="email" placeholder="Email" />
      @if (form.controls.email.dirty && form.controls.email.invalid) { <small>Enter a valid
email.</small> }

      <select formControlName="city">
```

```
        <option>Chennai</option>
        <option>Madurai</option>
        <option>Coimbatore</option>
      </select>

      <button type="submit" [disabled]="form.invalid">Save</button>
    </form>

    <p>Form status: {{ form.status }}</p>
    @if (saved()) { <pre>{{ saved() }}</pre> }
  `,
})
export class ReactiveFormComponent {
  private fb = inject(FormBuilder);

  // The form model lives in the class and is fully typed
  form = this.fb.nonNullable.group({
    name: ['', [Validators.required, Validators.minLength(3)]],
    email: ['', [Validators.required, Validators.email]],
    city: ['Chennai'],
  });

  saved = signal('');

  submit() {
    if (this.form.valid) {
      this.saved.set(JSON.stringify(this.form.getRawValue()));
    }
  }
}
```

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Fill in a name and an invalid email.
2. Correct the email and click **Save**.

```
--- Invalid email ---
Student Profile
Enter a valid email.
Chennai
Madurai
Coimbatore
Save
Form status: INVALID

--- After submit ---
Student Profile
Chennai
Madurai
Coimbatore
Save
Form status: VALID
{"name":"Meena","email":"meena@example.com","city":"Chennai"}
```

f. Real-World Example (English + தமிழ்)

English

Before printing wedding invitations you plan every field and rule with the designer first; the printer just follows. Reactive forms define the rules in code first, the template follows.

தமிழ்

திருமண அழைப்பிதழை அச்சடிக்கும் முன், மணமக்கள் பெயர், தேதி, மண்டபம் என்று எல்லா அமைப்பையும் முதலிலேயே வடிவமைப்பாளரிடம் திட்டமிட்டுக் கொடுப்போம்; அச்சகம் அதை அப்படியே பின்பற்றும். அச்சடித்த பின் காகிதத்தில் குழப்பம் வராது.

Reactive form-உம் அப்படியே: படிவத்தின் முழு விதிகளையும் (எந்தக் கட்டம், என்ன சரிபார்ப்பு) TypeScript-இல் முன்கூட்டியே எழுதிவிடுகிறோம்; HTML அதைப் பின்பற்றுகிறது. பெரிய படிவங்களுக்கும் சோதனை செய்வதற்கும் இது எளிது.

g. Pros

- Explicit and testable
- Strongly typed
- Great for dynamic forms (FormArray)
- Observable `valueChanges`

h. Cons

- More code than template-driven
- Steeper learning curve

i. Advanced / New Features

- Typed forms (v14)
- `FormRecord`
- `events` stream on controls (v18)
- Signal Forms experimental

j. Common Mistakes

- Forgetting to import `ReactiveFormsModule` (error: Can't bind to 'formGroup').
- Using `formControlName` outside an element that has `[formGroup]`.
- Reading `form.value` when you need disabled controls too; use `getRawValue()`.
- Putting validation logic in the template instead of using validators.

k. Interview Questions

Q1. What are reactive forms?
Forms whose model (FormGroup/FormControl) is created in the component class and bound to the template.

Q2. What is FormBuilder?
A helper service with shorthand methods (`group`, `control`, `array`) to build form models.

Q3. What is `valueChanges`?
An Observable that emits whenever a control or group value changes.

Q4. Why choose reactive forms?
They are explicit, strongly typed, scalable (dynamic fields with FormArray) and easier to unit-test.

l. Practice Exercises

1. Add a `phone` control that accepts exactly 10 digits using `Validators.pattern`.
2. Show a live preview card using `form.valueChanges` (hint: `toSignal`).
3. Add a `Reset` button that restores the default values with `form.reset()`.